Name - Ajinkya Kishan Dhotre
Roll no- 407I082
Class - BEIT
Title - Implementing Feedforward neural networks with Keras and TensorFlow
a. Import the necessary packages
b. Load the training and testing data (MNIST/CIFAR10)
c. Define the network architecture using Keras
d. Train the model using SGD
e. Evaluate the network
f. Plot the training loss and accuracy

```
import tensorflow as tf
```

```
from tensorflow import keras
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import random
%matplotlib inline
```

```
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-d
atasets/mnist.npz

11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 6s 0us/step
```

```
len(x_train)
```

```
60000
```

```
len(x_test)
```

```
10000
```

```
len(y_test)
```

```
10000
```

```
len(y_train)
```

```
60000
```

```
x_train.shape
```

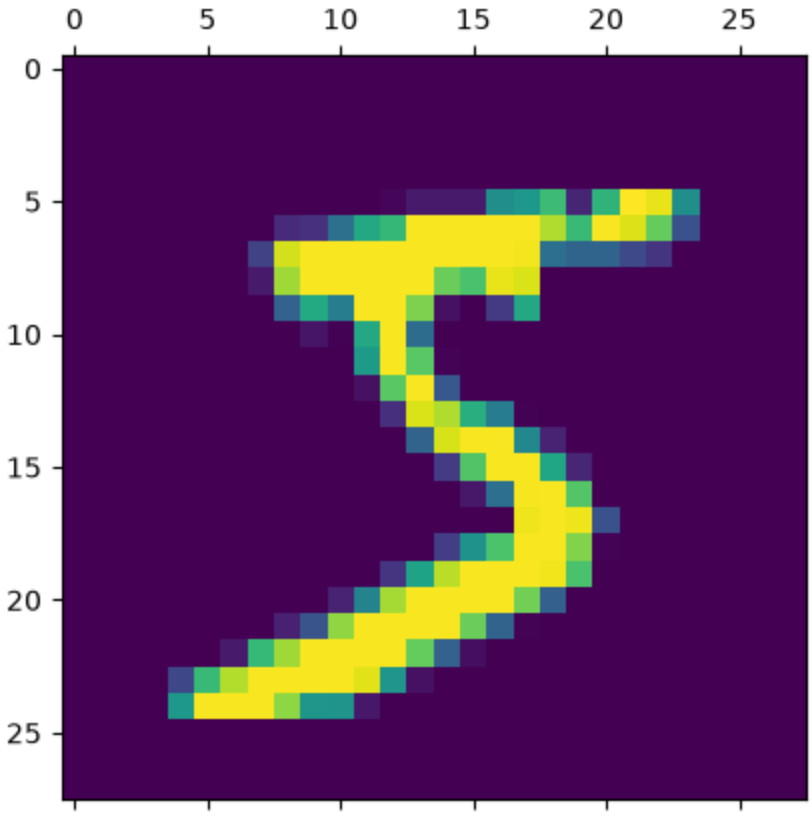
```
(60000, 28, 28)
```

```
x_train[0]
```

```
array([[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
...
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0]], dtype=uint8)
```

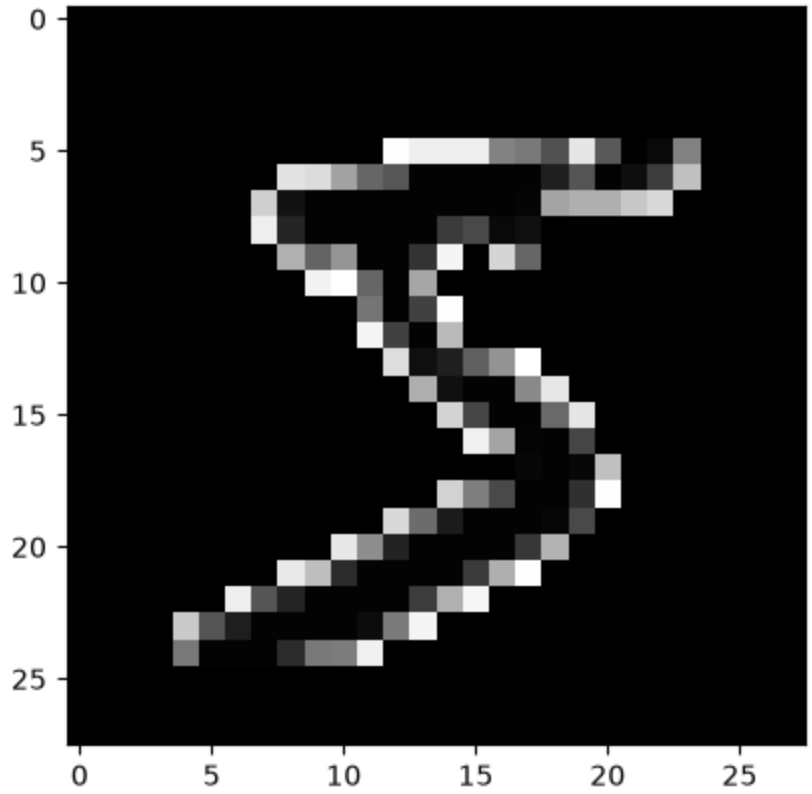
```
plt.matshow(x_train[0])
```

```
<matplotlib.image.AxesImage at 0x1599f05ade0>
```



```
plt.imshow(-x_train[0], cmap="gray")
```

```
<matplotlib.image.AxesImage at 0x159a08cc470>
```




```
x_train = x_train / 255
x_test = x_test / 255
```

```
x_train[0]
```

```
array([[0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        ],
...
       [0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        ]])
```

```
model = keras.Sequential([
keras.layers.Flatten(input_shape=(28, 28)),    #Input layer
keras.layers.Dense(128, activation="relu"),    #hidden layer abs
keras.layers.Dense(10, activation="softmax")   #output layer
])
model.summary()
```

```
C:\Users\AJINKYA\AppData\Local\Temp\claude\C--Users-AJINKYA-OneDrive-Deskt
op-SUBMISSION\71b4792e-b329-4612-8d3d-b3fb5437a28d\scratchpad\venv\Lib\sit
e-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not p
ass an `input_shape`/`input_dim` argument to a layer. When using Sequentia
l models, prefer using an `Input(shape)` object as the first layer in the
model instead.
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer="sgd",   # Stochastic Gradient Descent
loss="sparse_categorical_crossentropy",    # crossentropy reduces the loss
metrics=['accuracy'])
```

```
history=model.fit(x_train,y_train,validation_data=(x_test,y_test),epochs=10)
```

```
Epoch 1/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 24s 12ms/step - accuracy: 0.8359 - loss: 0.
6535 - val_accuracy: 0.9032 - val_loss: 0.3539

Epoch 2/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 22s 12ms/step - accuracy: 0.9070 - loss: 0.
3337 - val_accuracy: 0.9196 - val_loss: 0.2880

Epoch 3/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 27s 14ms/step - accuracy: 0.9200 - loss: 0.
2841 - val_accuracy: 0.9282 - val_loss: 0.2552

Epoch 4/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 26s 13ms/step - accuracy: 0.9293 - loss: 0.
2537 - val_accuracy: 0.9347 - val_loss: 0.2333

Epoch 5/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 43s 14ms/step - accuracy: 0.9354 - loss: 0.
2312 - val_accuracy: 0.9384 - val_loss: 0.2148

Epoch 6/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 25s 13ms/step - accuracy: 0.9400 - loss: 0.
2129 - val_accuracy: 0.9431 - val_loss: 0.2019

Epoch 7/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 42s 13ms/step - accuracy: 0.9436 - loss: 0.
1982 - val_accuracy: 0.9465 - val_loss: 0.1908

Epoch 8/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 14s 7ms/step - accuracy: 0.9481 - loss: 0.1
855 - val_accuracy: 0.9496 - val_loss: 0.1801

Epoch 9/10

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9503 - loss: 0.17
47 - val_accuracy: 0.9507 - val_loss: 0.1705

Epoch 10/10

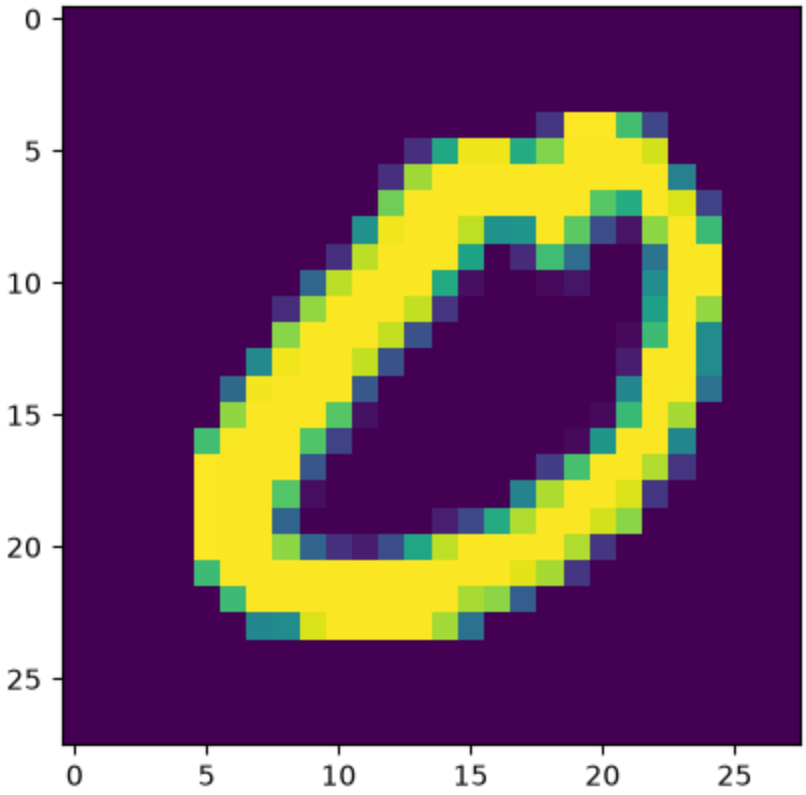
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9535 - loss: 0.16
46 - val_accuracy: 0.9534 - val_loss: 0.1625
```

```
test_loss,test_acc=model.evaluate(x_test,y_test)
print("Loss=%.3f" %test_loss)
print("Accuracy=%.3f" %test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9534 - loss: 0.1625

Loss=0.163
Accuracy=0.953
```

```
n=random.randint(0,9999)
plt.imshow(x_test[n])
plt.show()
```




```
#we use predict() on new data
predicted_value=model.predict(x_test)
print("Handwritten number in the image is= %d" %np.argmax(predicted_value[n]))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step

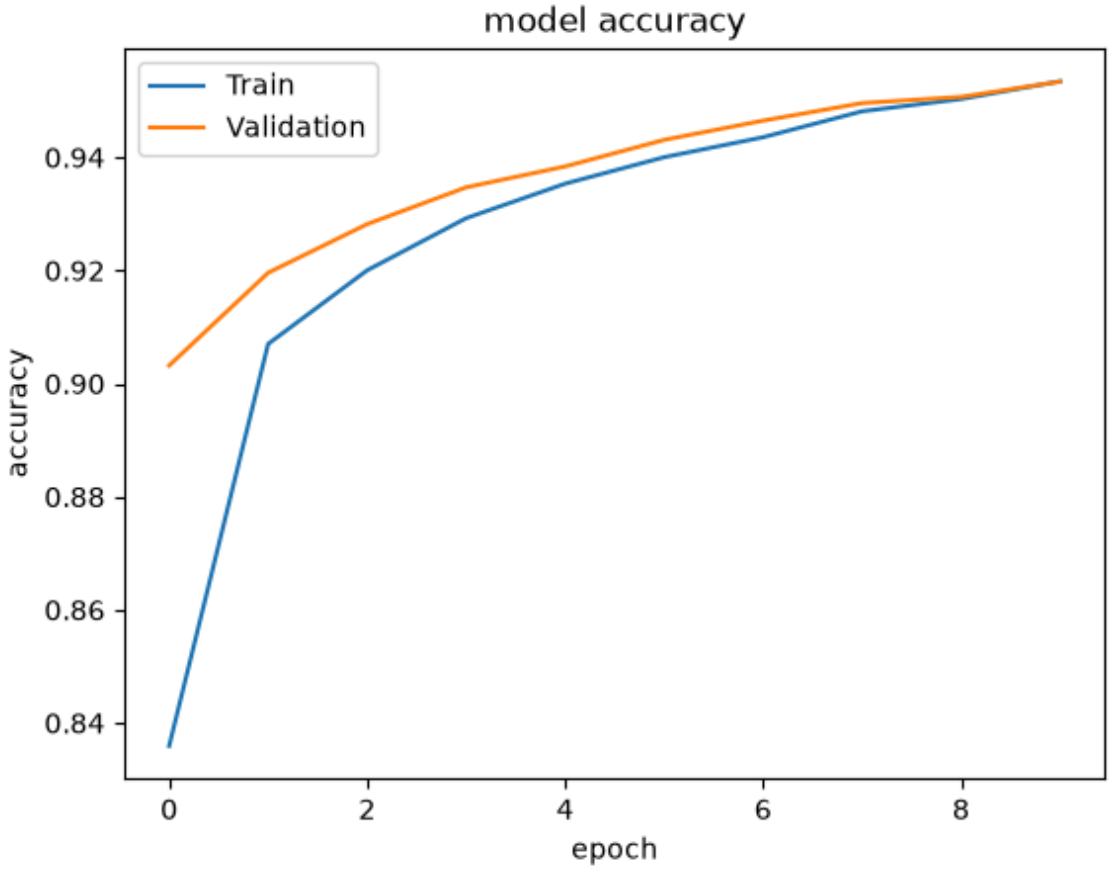
Handwritten number in the image is= 0
```

```
history.history??
```

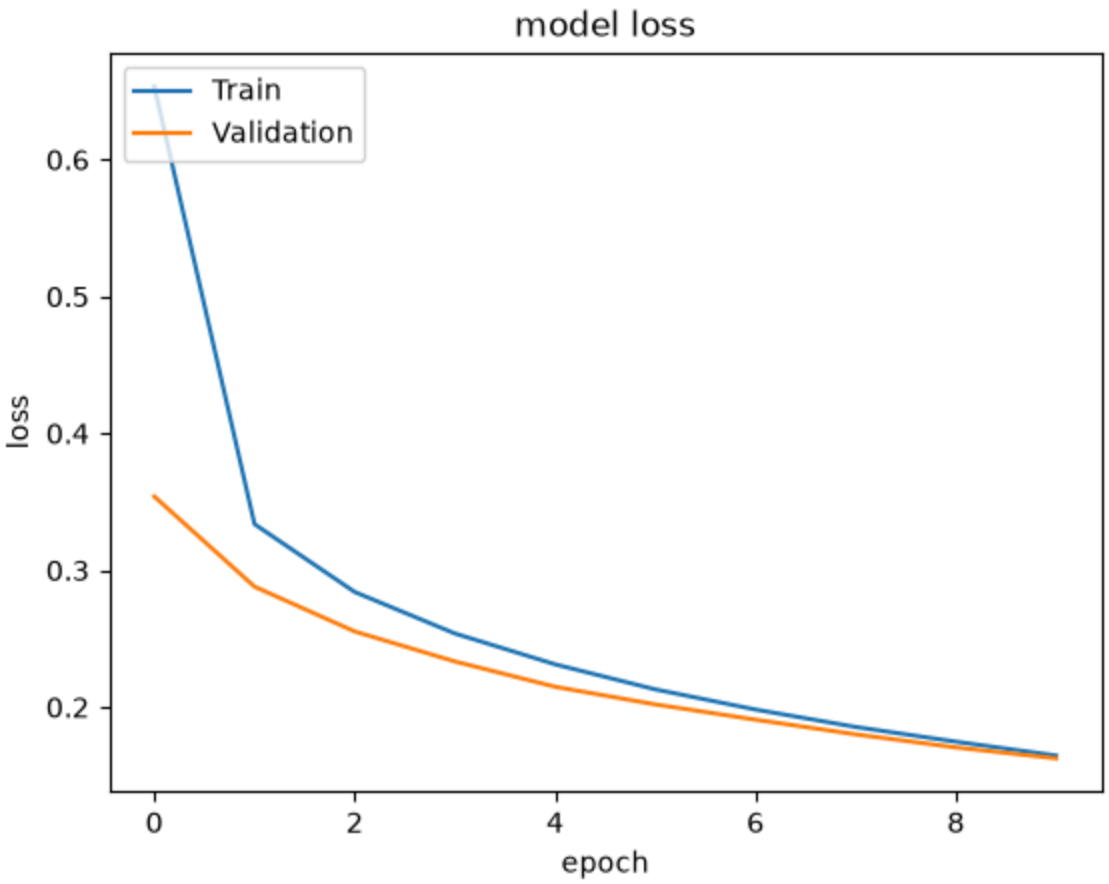
```
history.history.keys()
```

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
# Accuracy and loss on tarin data
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Training Loss and accuracy')
plt.ylabel('accuracy/Loss')
plt.xlabel('epoch')
plt.legend(['accuracy', 'val_accuracy','loss','val_loss'])
plt.show()
```

